Machine Learning II - Neurona M-P cancer

October 13, 2026



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub * URL: https://github.com/Exosphere-Idyllic/Machine-Learning

##Caso Práctico: Neurona de McCulloch y Pitts

Conjunto de datos Esta es una copia de los conjuntos de datos de UCI ML Breast Cancer Winconsin https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

Descipción de la muestra Las caracteristicas de netrada se calculan a partir de una imagen digitaizada de un aspirado de aguja fina (FNA) de una masa mamaria. Describen las caracteristicas de los npucles celulares presentes en la imagén. El plano de separación descrito anteriormente se obtuvo utilizando el m'petodo de árbol de multiples superficies (MSM-T) "COnstrucción de un árbol de decisión mediante programación lineal). El programa lineal utilizado para obtener el plano de separación en el espacio tridimensional es el que describe en [K.P Bennet y O. L Mangasarian]

Lectura de datos

[8]:
```
from sklearn.datasets import load_breast_cancer
cancer_mama= load_breast_cancer()
# Separamos Los datos de entra y los datos de salida
X= cancer_mama.data
Y=cancer_mama.target
```

[9]:
```
dir(cancer_mama)
```

[9]:
```
['DESCR',
 'data',
 'data_module',
```

```
 'feature_names',
 'filename',
 'frame',
 'target',
 'target_names']
```

[10]:
```
print(X)
```

```
[[1.799e+01 1.038e+01 1.228e+02 … 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 … 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 … 2.430e-01 3.613e-01 8.758e-02]
 …
 [1.660e+01 2.808e+01 1.083e+02 … 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 … 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 … 0.000e+00 2.871e-01 7.039e-02]]
```

Visualización del conjunto de datos

[11]:
```
import pandas as pd
df=pd.DataFrame(X, columns=cancer_mama.feature_names)
df
```

[11]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
..	…	…	…	…	…	
564	21.56	22.39	142.00	1479.0	0.11100	
565	20.13	28.25	131.20	1261.0	0.09780	
566	16.60	28.08	108.30	858.1	0.08455	
567	20.60	29.33	140.10	1265.0	0.11780	
568	7.76	24.54	47.92	181.0	0.05263	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.30010	0.14710	0.2419	
1	0.07864	0.08690	0.07017	0.1812	
2	0.15990	0.19740	0.12790	0.2069	
3	0.28390	0.24140	0.10520	0.2597	
4	0.13280	0.19800	0.10430	0.1809	
..	…	…	…	…	
564	0.11590	0.24390	0.13890	0.1726	
565	0.10340	0.14400	0.09791	0.1752	
566	0.10230	0.09251	0.05302	0.1590	
567	0.27700	0.35140	0.15200	0.2397	
568	0.04362	0.00000	0.00000	0.1587	

```
     mean fractal dimension  …  worst radius  worst texture  \
0                   0.07871  …        25.380          17.33
1                   0.05667  …        24.990          23.41
2                   0.05999  …        23.570          25.53
3                   0.09744  …        14.910          26.50
4                   0.05883  …        22.540          16.67
..                      …  …           …             …
564                 0.05623  …        25.450          26.40
565                 0.05533  …        23.690          38.25
566                 0.05648  …        18.980          34.12
567                 0.07016  …        25.740          39.42
568                 0.05884  …         9.456          30.37

     worst perimeter  worst area  worst smoothness  worst compactness  \
0             184.60      2019.0           0.16220            0.66560
1             158.80      1956.0           0.12380            0.18660
2             152.50      1709.0           0.14440            0.42450
3              98.87       567.7           0.20980            0.86630
4             152.20      1575.0           0.13740            0.20500
..               …         …               …                 …
564           166.10      2027.0           0.14100            0.21130
565           155.00      1731.0           0.11660            0.19220
566           126.70      1124.0           0.11390            0.30940
567           184.60      1821.0           0.16500            0.86810
568            59.16       268.6           0.08996            0.06444

     worst concavity  worst concave points  worst symmetry  \
0             0.7119                0.2654          0.4601
1             0.2416                0.1860          0.2750
2             0.4504                0.2430          0.3613
3             0.6869                0.2575          0.6638
4             0.4000                0.1625          0.2364
..               …                   …             …
564           0.4107                0.2216          0.2060
565           0.3215                0.1628          0.2572
566           0.3403                0.1418          0.2218
567           0.9387                0.2650          0.4087
568           0.0000                0.0000          0.2871

     worst fractal dimension
0                    0.11890
1                    0.08902
2                    0.08758
3                    0.17300
4                    0.07678
..                       …
564                  0.07115
```

```
565                0.06637
566                0.07820
567                0.12400
568                0.07039

[569 rows x 30 columns]
```

[12]:
```
print(Y)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

División del conjunto de datos

[13]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df, Y, stratify = Y)
print("Tamaño de entrenamiento ", len(X_train))
print("Tamalo de pruebas", len(X_test))
```

```
Tamaño de entrenamiento  426
Tamalo de pruebas 143
```

Implementación de un aMP Neurona más avanzada

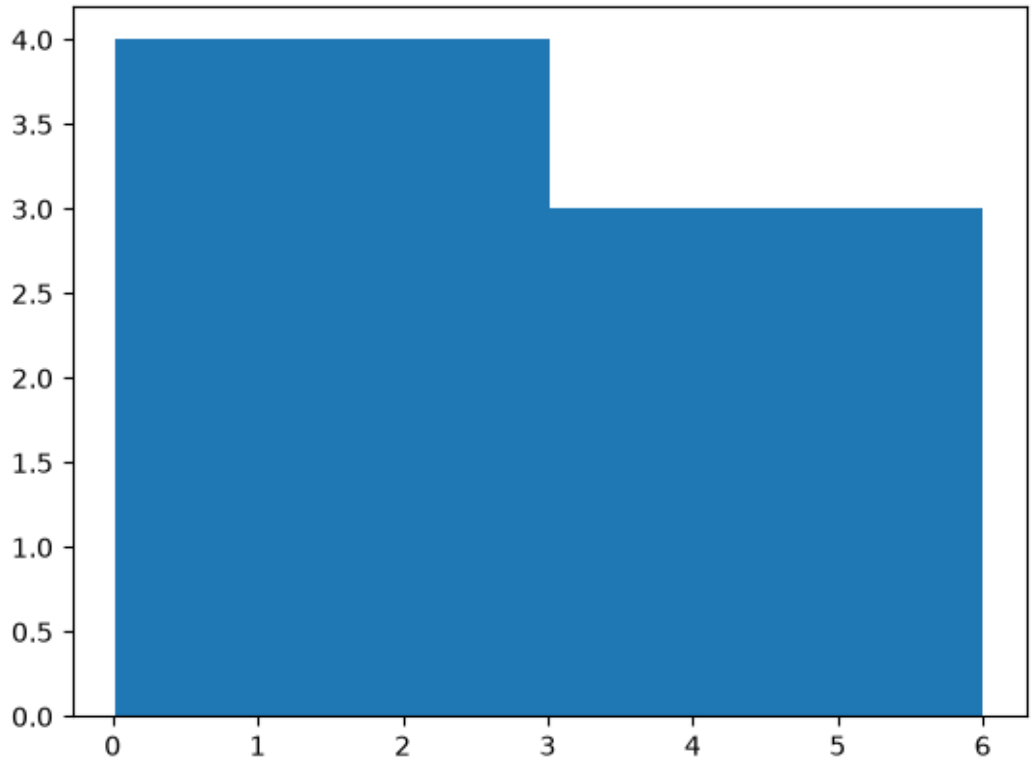
[16]:
```
# importamos la biblioteca de numpy
import numpy as np
from sklearn.metrics import accuracy_score
class MpNeurona:
# creamos nuestro constructor
    def __init__(self):
        self.threshold=None
# Creamos una función par el modelo
    def model(self, x):
#input: [1,0,1,0][x1,x2,...xn]
        z=sum(x)
```

```
        return (z >= self.threshold)
#Creamos la predicción
    def predict(self, X):
# input:[[1,0,1,0][1,0,1,1]]
        Y=[]
        for x in X:
            result=self.model(x)
            Y.append(result)
        return np.array(Y)
# función de entrenamiento para e lthreshold
    def fit(self, X, Y):
        accuracy={}
#Seleccionamos un threshold entre el # de caracteristicas de entrada
        for th in range(X.shape[1]+1):
            self.threshold = th
            Y_pred = self.predict(X)
            accuracy[th]=accuracy_score(Y_pred, Y)
#Seleccionamos el threshold que mejores resultado proporciona
        self.threshold = max(accuracy, key=accuracy.get)
```

Los datos de entreeda siguen teniendo porblemas debido a que nuestro conjunto de datos las caracteristicas de entreda reciben valores continuos, sin embargo, la neurona MP solo procesa características de netrada con valores binarios

[17]:
```
#Para transformar un valor a binario
import matplotlib.pyplot as plt
print(pd.cut([0.04,0.3,4,5,6,0.02,0.6], bins=2, labels=[0,1]))
plt.hist([0.04,0.3,4,5,6,0.02,0.6], bins=2)
plt.show()
```

```
[0, 0, 1, 1, 1, 0, 0]
Categories (2, int64): [0 < 1]
```



[18]:
```
#Trasformarmamos las caracteristicas de entrada a un valor binario
X_train_bin= X_train.apply(pd.cut, bins=2, labels=[1,0])
X_test_bin = X_test.apply(pd.cut, bins=2, labels=[1,0])
X_train_bin
```

[18]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
10	1	0	1	1	1	
159	1	1	1	1	1	
39	1	1	1	1	1	
92	1	1	1	1	1	
270	1	1	1	1	1	
..	…	…	…	…	…	
136	1	1	1	1	1	
158	1	1	1	1	1	
265	0	0	0	0	1	
182	1	1	1	1	1	
191	1	1	1	1	1	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
10	1	1	1	1	
159	1	1	1	1	
39	1	1	1	1	

```
92                 1              1                    1             1
270                1              1                    1             1
..               …              …                    …             …
136                1              1                    1             1
158                1              1                    1             1
265                1              1                    1             1
182                1              1                    1             1
191                1              1                    1             1

    mean fractal dimension  … worst radius worst texture worst perimeter  \
10                       1  …            1             0               1
159                      1  …            1             1               1
39                       1  …            1             1               1
92                       1  …            1             1               1
270                      1  …            1             1               1
..                     …  …          …           …             …
136                      1  …            1             1               1
158                      1  …            1             1               1
265                      1  …            0             0               0
182                      1  …            1             0               1
191                      1  …            1             1               1

    worst area worst smoothness worst compactness worst concavity  \
10           1                1                 1               1
159          1                1                 1               1
39           1                0                 1               1
92           1                1                 1               1
270          1                1                 1               1
..         …              …                 …             …
136          1                1                 1               1
158          1                1                 1               1
265          0                1                 1               1
182          1                1                 1               1
191          1                1                 1               1

    worst concave points worst symmetry worst fractal dimension
10                     1              1                       1
159                    1              1                       1
39                     0              1                       1
92                     1              1                       1
270                    1              1                       1
..                   …              …                       …
136                    1              1                       1
158                    1              1                       1
265                    0              1                       1
182                    0              1                       1
191                    1              1                       1
```

```
[426 rows x 30 columns]
```

[19]:
```
# instaciamos el modelo MP
np_neurona = MpNeurona()
#Encontramos el threshold optimo
np_neurona.fit(X_train_bin.to_numpy(), y_train)
# threshold optimo seleccionado
np_neurona.threshold
```

[19]:
```
27
```

[20]:
```
# Realizamos la predicción par alos datos de prueba
Y_pred=np_neurona.predict(X_test_bin.to_numpy())
```

[21]:
```
Y_pred
```

[21]:
```
array([False,  True,  True,  True,  True,  True,  True, False,  True,
        True,  True,  True,  True, False,  True,  True, False,  True,
       False, False,  True,  True,  True,  True,  True,  True,  True,
       False, False,  True, False, False, False, False, False,  True,
        True,  True, False,  True,  True,  True, False, False,  True,
        True, False,  True,  True,  True, False, False, False,  True,
        True, False, False,  True,  True, False, False,  True,  True,
        True,  True, False, False,  True,  True,  True, False, False,
        True,  True,  True, False,  True, False,  True,  True,  True,
        True,  True, False,  True,  True,  True,  True,  True,  True,
        True,  True,  True, False,  True,  True, False,  True, False,
        True, False, False,  True, False,  True, False, False,  True,
       False,  True, False,  True,  True,  True, False, False,  True,
       False,  True,  True, False,  True, False,  True,  True, False,
        True,  True,  True, False,  True, False,  True,  True, False,
        True,  True,  True, False,  True,  True,  True,  True])
```

[22]:
```
# Calculaculamos la exactitud de nuestra predicción
accuracy_score(y_test, Y_pred)
```

[22]:
```
0.8321678321678322
```